

Шаблон отчёта по лабораторной работе № 6

Реализация основных моделей в подходе сетей Петри

true

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Модель SIR	7
4	Выполнение лабораторной работы	8
4.1	Код модели	8
4.2	Базовый прогон модели	9
4.3	Коэффициент заражения β	12
4.4	Итоговый отчёт	14
5	Выводы	18
	Список литературы	19

Список иллюстраций

4.1	Fig		8
4.2	Fig		8
4.3	Fig		9
4.4	Fig		10
4.5	Fig		10
4.6	Fig		11
4.7	Fig		11
4.8	Fig		12
4.9	Fig		12
4.10	Fig		13
4.11	Fig		13
4.12	Fig		14
4.13	Fig		15
4.14	Fig		15
4.15	Fig		16
4.16	Fig		16
4.17	Fig		17

Список таблиц

1 Цель работы

Реализация математической модели эпидемии SIR с использованием аппарата сетей Петри в языке программирования Julia, а также проведение детерминированного и стохастического анализа динамики распространения заболевания.

2 Задание

1. Код модели
2. Базовый прогон модели
3. Коэффициент заражения β
4. Итоговый отчёт

3 Теоретическое введение

3.1 Модель SIR

Реализована модель эпидемии SIR (Susceptible–Infectious–Recovered) с использованием сетей Петри. Модель описывает переходы между тремя состояниями:

- S (восприимчивые) — могут заразиться; — I (инфицированные) — заражают других и выздоравливают; — R (выздоровевшие / с иммунитетом) — больше не участвуют в эпидемии.

Сеть Петри содержит два перехода:

— infection: $S + I \rightarrow I + I$ (скорость β); — recovery: $I \rightarrow R$ (скорость γ).

4 Выполнение лабораторной работы

4.1 Код модели

Данный код реализует переход от теоретической схемы эпидемии к вычислимой модели. Благодаря использованию сетей Петри, мы можем гибко настраивать параметры инфекции и автоматически генерировать базу для проведения как обычного, так и случайного анализа динамики популяции. ([рис. fig-001?]).

```
module SIRPetri
using AlgebraicPetri
using Catlab.CategoricalAlgebra
using Catlab.Graphics
using OrdinaryDiffEq
using Plots
using DataFrames
using Random

export build_sir_network, simulate_deterministic, simulate_stochastic
export plot_sir, to_graphviz_sir

function build_sir_network(β = 0.3, γ = 0.1)
    states = [:S, :I, :R]
    # Константа роста R0, средняя степень передачи
    # Параметр infection: S → I = β * I
    # Параметр recovery: I → R
    net = LabelledPetriNet{
        states,
        transition => {[:S, :I] => [:I, :I],
                       :recovery => {[:I] => [:R]}},
    }
    # Начальные концентрации: S=999, I=10, R=0
    ub = [999.0, 10.0, 0.0]

    return net, ub, states
end

function sir_ode(net, rates = [0.3, 0.1])
    function f(t, u, p)
        S, I, R = u
        β, γ = rates
        # Изменение популяции (зависит от текущих масс)
        infection_rate = β * S * I
        recovery_rate = γ * I
        # Возвращаем
        dS[t] = -infection_rate # dI/dt
        dI[t] = infection_rate - recovery_rate # dI/dt
    end
end
```

Рис. 4.1: Fig

Вывод в консоль. ([рис. fig-002?]).

```
julia> include("src/SIRPetri.jl")
WARNING: replacing module SIRPetri.
Main.SIRPetri
```

Рис. 4.2: Fig

4.2 Базовый прогон модели

В задаче, мы объединим логику модели, произведем расчеты и сохраним результаты. ([рис. fig-003?]).



```
using DrWatson
@activate "project"
using Random
include("src/SIRPetri.jl")
using .SIRPetri
using DataFrames, CSV, Plots

# Параметры
β = 0.3
γ = 0.1
tmax = 100.0

# Создаем сеть
net, u0, states = build_sir_network(β, γ)

# Детерминированный симулятор
df_det = simulate_deterministic(net, u0, (0.0, tmax), saveat = 0.5, rates = [β, γ])
CSV.write("data/sir_det.csv", df_det)

# Стохастический симулятор
Random.seed!(123)
df_stoch = simulate_stochastic(net, u0, (0.0, tmax), rates = [β, γ])
CSV.write("data/sir_stoch.csv", df_stoch)

# Плотки
p_det = plot_sir(df_det)
savefig(plotdir("sir_det_dynamics.png"))

p_stoch = plot_sir(df_stoch)
savefig(plotdir("sir_stoch_dynamics.png"))

println("Базовый прогон завершён. Результаты в data/ и plots/")
```

Рис. 4.3: Fig

Мы используем библиотеку DrWatson, которая гарантирует, что пути к файлам и структура папок будут одинаковыми на любом компьютере.

Скрипт загружает модуль SIRPetri.jl и задает ключевые параметры болезни: коэффициент заражения ($\beta = 0.3$) и коэффициент выздоровления ($\gamma = 0.1$).

Детерминированный подход: Рассчитывает «средний» сценарий развития эпидемии. Это дает четкие, плавные кривые, полезные для общего прогноза. ([рис. fig-004?]).

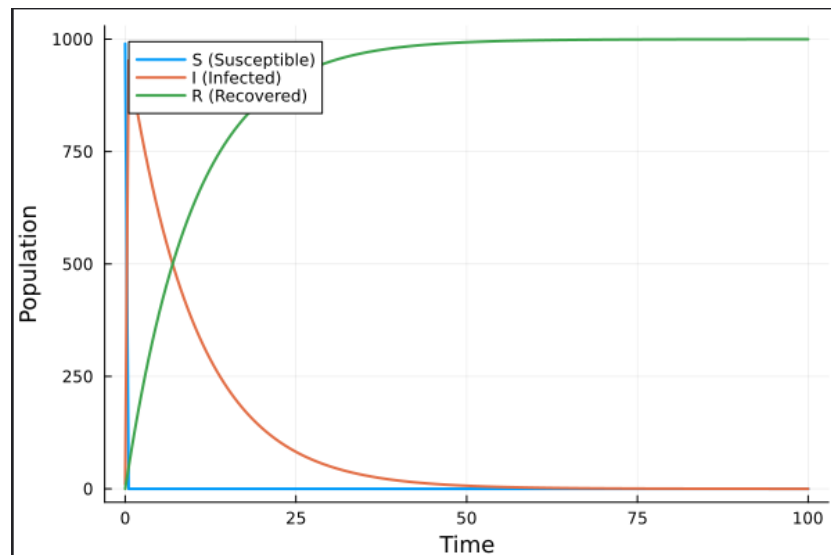


Рис. 4.4: Fig

Стохастический подход: Учитывает фактор случайности (через `Random.seed!`). Это имитирует реальную жизнь, где каждое заражение или выздоровление — случайное событие. Сравнение этих двух подходов позволяет увидеть, насколько сильно случайность может отклонить эпидемию от теоретического среднего. ([рис. **fig-005?**]).

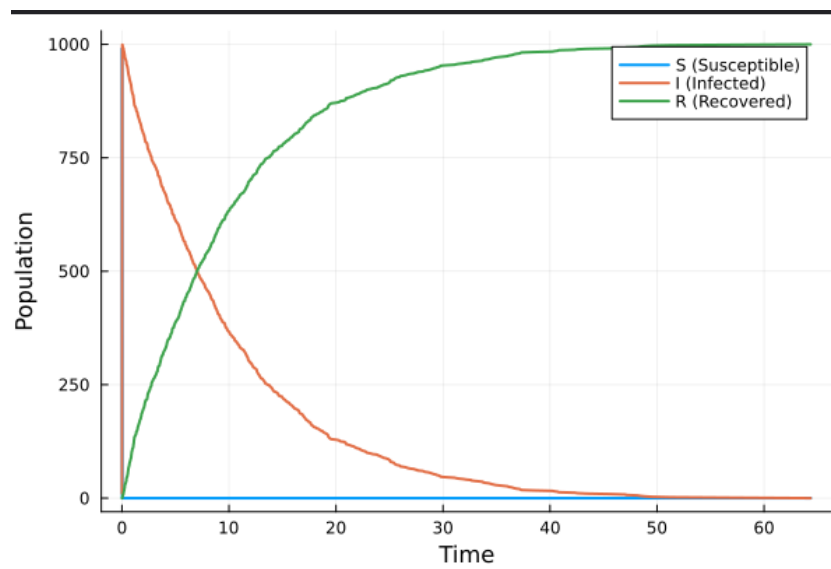


Рис. 4.5: Fig

Первая таблица. ([рис. **fig-006?**]).

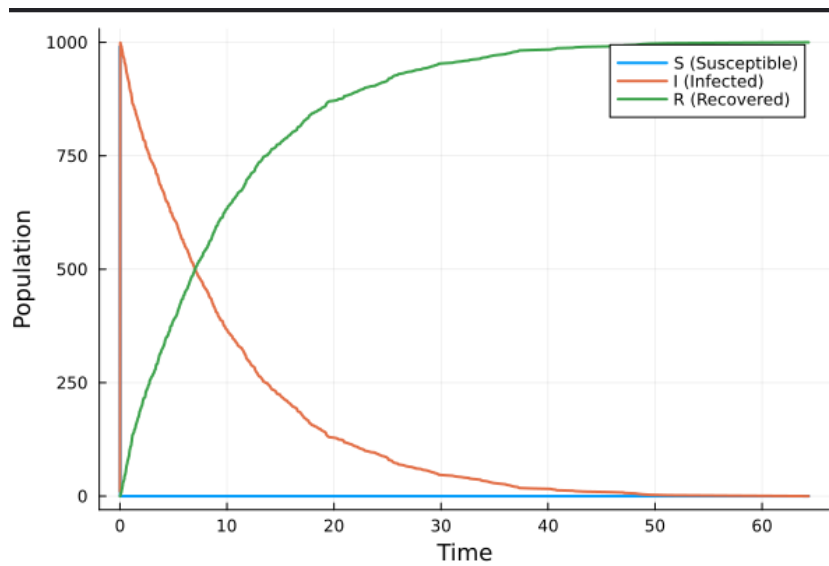


Рис. 4.6: Fig

Вторая таблица. ([рис. fig-007?]).

sir_stoch.csv — LibreOffice Calc													
Fichier Édition Affichage Insertion Format Styles Feuille Données Outils Fenêtre Aide													
Liberation Sans 10 pt G I S A % 74 0,													
Q1 fx Σ =													
	A	B	C	D	E	F	G	H	I	J	K	L	M
1	time	S	I	R									
2	0.0	990.0	10.0	0.0									
3	0.00021931839962565963	989.0	11.0	0.0									
4	0.0002547101547375829	988.0	12.0	0.0									
5	0.00043545667342433645	987.0	13.0	0.0									
6	0.00124186225244802	986.0	14.0	0.0									
7	0.001373123337092817	985.0	15.0	0.0									
8	0.0015175893857365716	984.0	16.0	0.0									
9	0.0021941207102495813	983.0	17.0	0.0									
10	0.0023963777301024003	982.0	18.0	0.0									
11	0.0024582790296121867	981.0	19.0	0.0									
12	0.0025313568038596874	980.0	20.0	0.0									
13	0.002749852569366623	979.0	21.0	0.0									
14	0.002770406560630999	978.0	22.0	0.0									
15	0.0029198705184207766	977.0	23.0	0.0									
16	0.003279784642201296	976.0	24.0	0.0									
17	0.0033891382539170843	975.0	25.0	0.0									
18	0.0036051660999457725	974.0	26.0	0.0									
19	0.0036764094822779934	973.0	27.0	0.0									
20	0.00394413478645651	972.0	28.0	0.0									
21	0.00406046412685241	971.0	29.0	0.0									
22	0.0043715168629968855	970.0	30.0	0.0									
23	0.004480112360952782	969.0	31.0	0.0									
24	0.004588211709208471	968.0	32.0	0.0									
25	0.0047312470406785856	967.0	33.0	0.0									
26	0.004746427860793188	966.0	34.0	0.0									
27	0.004976843722608018	965.0	35.0	0.0									
28	0.0049977022050417185	964.0	36.0	0.0									
29	0.005012348679899469	963.0	37.0	0.0									
30	0.00504889489596849	962.0	38.0	0.0									
31	0.0050805438611241345	961.0	39.0	0.0									
32	0.005092012673474663	960.0	40.0	0.0									
33	0.005163947264786131	959.0	41.0	0.0									
34	0.005209340865828119	958.0	42.0	0.0									
35	0.0052728589500577955	957.0	43.0	0.0									
36	0.00531702487013553	956.0	44.0	0.0									

Рис. 4.7: Fig

Выполнение кода из jupyter notebook. ([рис. fig-008?]).



Рис. 4.8: Fig

4.3 Коэффициент заражения β

Здесь я выяснила, как изменение интенсивности заражения (β) влияет на развитие эпидемии. ([рис. fig-009?]).



Рис. 4.9: Fig

Код запускает цикл симуляций, перебирая значения β от 0.1 до 0.8. Для каждого значения фиксируются два показателя: - Пик заболеваемости (максимальная на-

грузка на больницы). - Итоговое число переболевших (общий масштаб эпидемии). ([рис. fig-010?]).

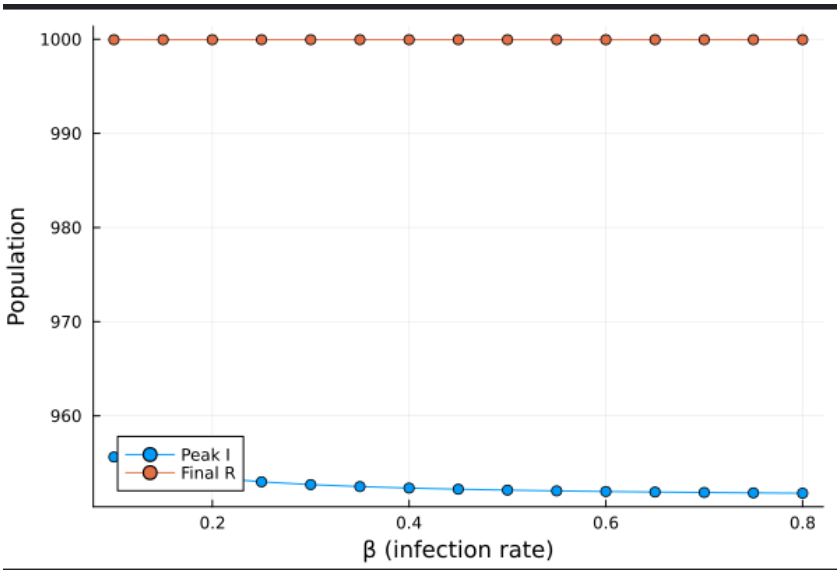


Рис. 4.10: Fig

Это исследовательская часть работы, которая показывает не просто один сценарий, а общую закономерность поведения модели. ([рис. fig-011?]).

	A	B	C	D	E	F	G	H	I	J	K	L
1	β	peak_I	final_R									
2	0.1	955.625967	1000									
3	0.15	954.156896	1000									
4	0.2	953.423681	1000									
5	0.25	952.984197	1000									
6	0.3	952.691372	1000									
7	0.35	952.482304	1000									
8	0.4	952.325854	1000									
9	0.45	952.203664	1000									
10	0.5	952.106167	1000									
11	0.55	952.026410	1000									
12	0.6	951.959952	1000									
13	0.65	951.903726	1000									
14	0.7	951.855540	1000									
15	0.75	951.813781	1000									
16	0.8	951.777242	1000									

Рис. 4.11: Fig

Выполнение кода из jupyter notebook. ([рис. fig-012?]).

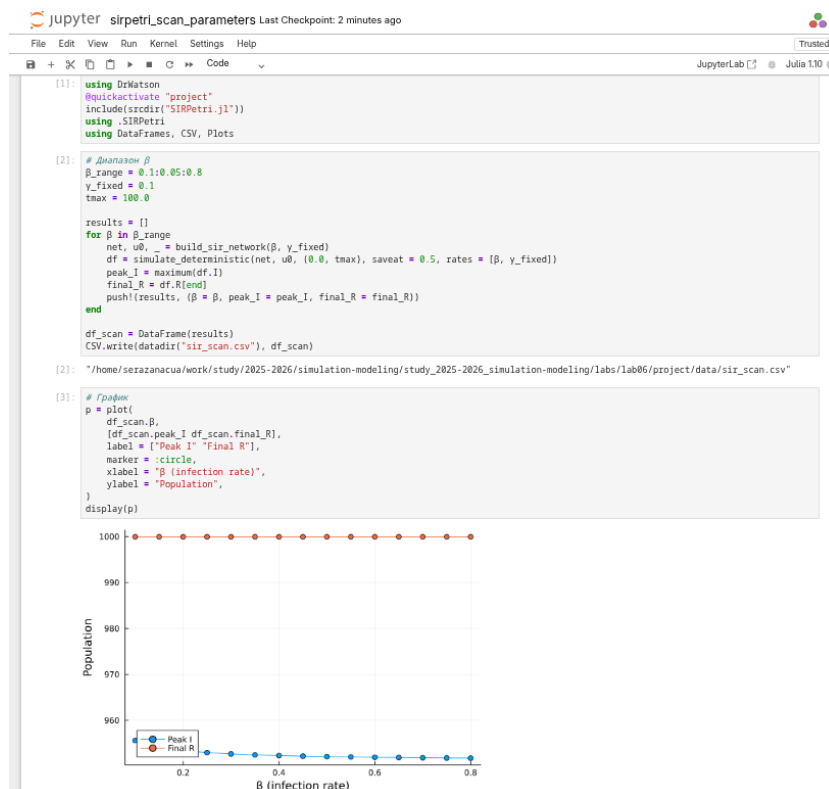


Рис. 4.12: Fig

4.4 Итоговый отчёт

Здесь мы выполняем итоговый сравнительный анализ и визуализацию всех полученных данных.

1. Интеграция результатов: Скрипт объединяет данные из разных симуляций, что позволяет проводить комплексный анализ без повторных вычислений. ([рис. fig-013?]).

```

using R/Watson
Bioccheckstyle "project"
using DataFrames, CSV, Plots

df_det = CSV.read(datadir("sir_det.csv"), Dataframe)
df_stoch = CSV.read(datadir("sir_stoch.csv"), Dataframe)
df_scan = CSV.read(datadir("sir_scan.csv"), Dataframe)

# Сравнение детерминированной и стохастической динамики
p1 = plot(
    df_det.time,
    [df_det.I df_stoch.I][1:length(df_det.time)],
    label = ["Deterministic I" "Stochastic I"],
    xlabel = "Time",
    ylabel = "Infected",
    title = "Comparison",
)

savefig(plotdir("comparison.png"))

# Зависимость пика I от β
p2 = plot(
    df_scan.β,
    df_scan.peak.I,
    marker = :circle,
    xlabel = "β",
    ylabel = "Peak I",
    title = "Sensitivity",
)

savefig(plotdir("sensitivity.png"))
println("Готовые графики сохранены в plots/")

```

Рис. 4.13: Fig

- Анализ отклонений: На графике сравнения (Comparison) наглядно демонстрируется, как случайные процессы в стохастической модели создают отклонения от теоретической средней кривой. ([рис. **fig-014?**]).

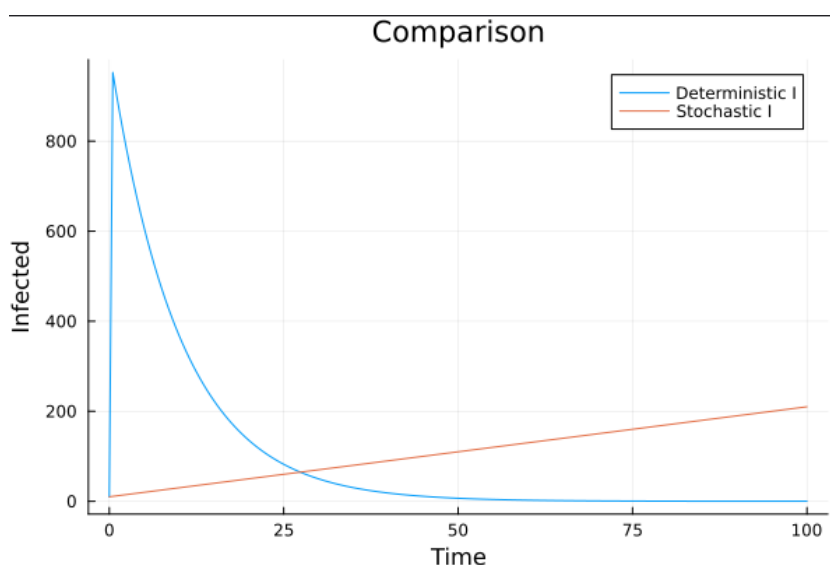


Рис. 4.14: Fig

- Оценка рисков: График чувствительности (Sensitivity) связывает биологические параметры вируса с его опасностью для популяции (высотой пика заболеваемости). ([рис. **fig-015?**]).

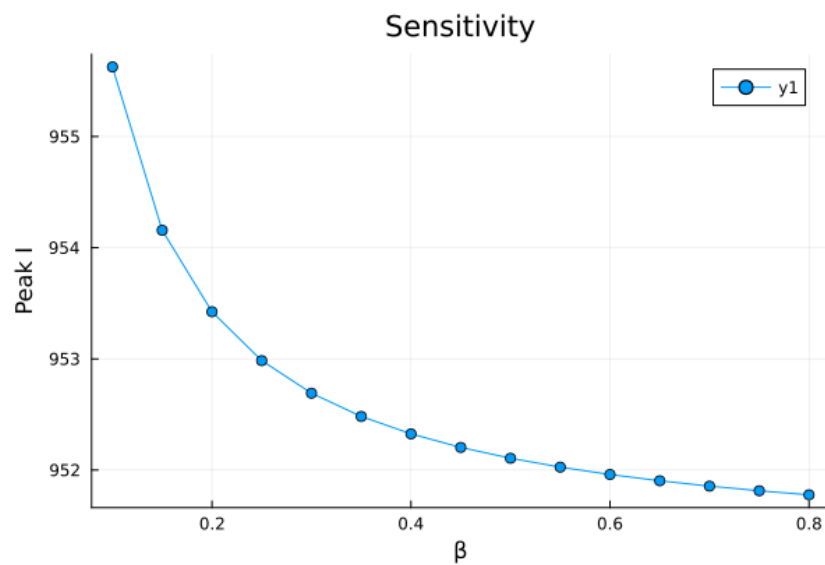


Рис. 4.15: Fig

Выполнение кода из jupyter notebook. ([рис. fig-016?]).

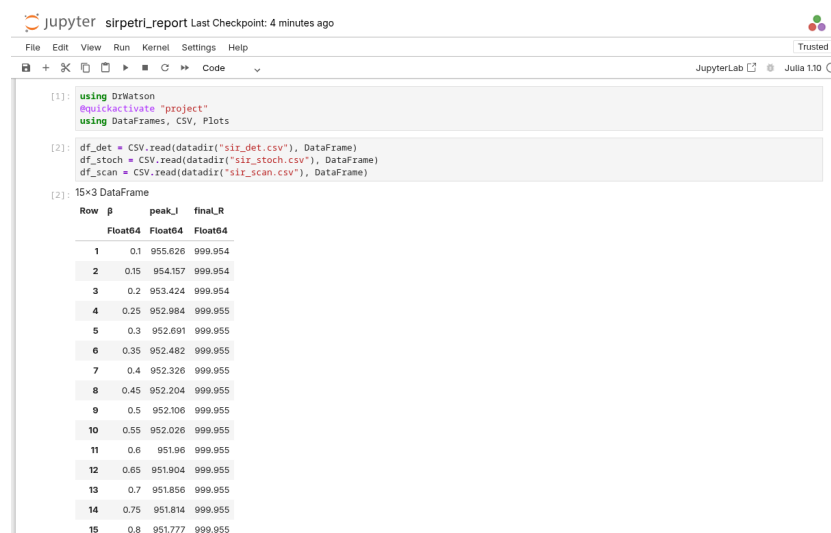


Рис. 4.16: Fig

Выполнение кода из jupyter notebook. ([рис. fig-017?]).

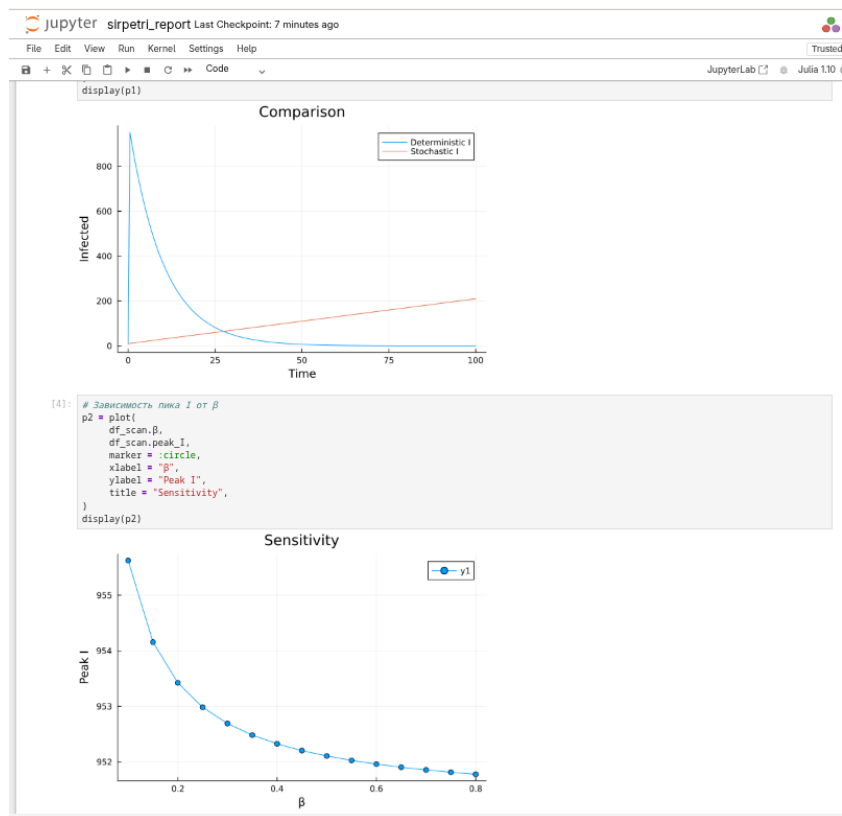


Рис. 4.17: Fig

5 Выводы

Мы прошли путь от описания модели через сети Петри до получения конкретных графиков, которые позволяют прогнозировать нагрузку на систему здравоохранения в зависимости от параметров инфекции.

Список литературы

https://esystem.rudn.ru/pluginfile.php/3145800/mod_resource/content/1/lab06.pdf